Invoice anomalies

Ranks a cost department's invoices so the wrong ones are looked at first, and proves on measured evidence whether that is worth doing.

Tutorial Â· Reference Â· Sources Â· Reading order

The invoices are filed at last. Some of them are wrong.

Moty imports cosmetics packaging from Asia: bottles, caps, pumps, labels. Its plant is in Louveira, a real city in SÃ£o Paulo state; Moty itself is invented. Factories, shipping lines, forwarders, a customs broker, a terminal and a truck company bill it.

You work in **Production and Logistics Cost Management**. The earlier piece of work got every invoice out of the mail and into rows, so you finally know **what every part cost to land at the plant**. It also wrote down, apart from the rows, *which invoices it had billed wrong on purpose*. That answer key is the only reason anything here can be scored.

Now the problem changes. The run holds **1,003 invoices and 35 real errors**, with another 46 built to draw a false positive. In an average week that is 24 invoices and 1 error.

The department does not pay these invoices. **A wrong invoice is a wrong number in the cost report**, and nobody reads a thousand pages looking for 35. So the invoices are ranked, worst first, and each one that is checked ends up confirmed, dismissed, or still open.

```
%%{init: {"flowchart": {"subGraphTitleMargin": {"top": 6, "bottom": 24},
"nodeSpacing": 30, "rankSpacing": 70}}}%%
flowchart LR
    subgraph Before["<b>Checking by hand</b><br/>WITHOUT a model"]
        B1["<b>1,003 invoices</b><br/>nobody can read them all"]
        B2["<b>35 errors</b><br/>found by chance, or never"]
        B3["<b>The cost report</b><br/>carries whatever was billed"]
    end
    subgraph After["<b>Checking by rank</b><br/>WITH a model on duty"]
        A1["<b>Ranked weekly</b><br/>worst first"]
        A2["<b>Each check decided</b><br/>confirmed, dismissed, open"]
        A3["<b>Every verdict</b><br/>trains the next model"]
    end
    B1 --> A1
    B2 --> A2
    B3 --> A3

    classDef before fill:#3b1f24,stroke:#d1495b,color:#f2dede
    classDef after fill:#16332a,stroke:#2e9e6b,color:#dff2e8
    class B1,B2,B3 before
    class A1,A2,A3 after
    style Before fill:transparent,stroke:#d1495b,stroke-dasharray:4 3
    style After fill:transparent,stroke:#2e9e6b
```

Why it matters:

- A typical company pays **1.5% of its disbursements** twice or in error [[1]](docs/sources.md).
- **3 to 6 of every 100** freight invoices carry an **error** [[2]](docs/sources.md).

Every number in this project, and where it comes from: sources.

How the work is organised

The project follows **CRISP-DM**, the cross-industry standard process for data mining (Chapman et al., 2000): six phases, run in order and revisited as often as the work demands.

Phase	Here
Business understanding	a wrong invoice is a wrong number in the cost report; the department can check only a few a week
Data understanding	1,003 invoices, 35 errors, 46 false positives, and the records they are checked against
Data preparation	one row per invoice, built only from what was knowable on its own date
Modeling	the checks a clerk makes by hand, then models trained on the errors already found
Evaluation	trained on earlier weeks, tested on later ones: confusion matrix, precision, recall, F1, the precision-recall curve and its average precision, against a baseline of the clerk's own checks
Deployment	the ranking is published, each verdict is recorded, and the model is retrained and replaced on that evidence

Deployment is not the end. CRISP-DM is a cycle, and this project runs it continuously: the verdicts become labels, the labels train the next model, and the next model is measured before it replaces the one on duty.


Sketch of the four panels: confusion matrix, precision against recall, errors caught per invoice checked, and errors caught by kind

A sketch. Every number in it is made up, and it is here only to show the shape of the four panels the results will be reported in.

What is actually in the run

The answer key names each error and, beside it, the correct invoices built to be mistaken for one. Across its 1,044 rows it holds 37 errors and 48 false positives.

Error	Real	False positives
a vendor billing with no order behind it	10	2
a freight charge not on the rate card	10	20
a duty at a rate not in force	5	2
a broker fee above its contract	5	6
the same bill again under a new number	3	11
a price above the contract	2	0
more billed than the dock received	2	0
an account or sender the master does not hold	0	6
a round adjustment to the total	0	1
a delivery split under the approval limit	0	0
total	**37**	**48**

Counted once per invoice number, the run holds 1,003 invoices, 35 errors and 46 false positives: 41 numbers were issued twice, revised or resent, and their repeats carry no new error.

Three kinds never happened in these 9Â½ months. **Two of them appear only as false positives**, which is the harder case: a bank change that was announced and recorded, and a fixed fee that lands on a round number. The model will be asked about them without ever having seen the real thing.

Results

The clerk's twelve checks come out ahead. On the errors held back from July to October, the checks caught every one; no model caught more than five. But every interval overlaps, so the comparison is not decided: the checks are ahead on the estimate, not separated from the models by the evidence.

Model	Precision	Recall	Average precision
Clerk's checks (`rules`)	0.15 (0.07 to 0.23)	1.00 (1.00 to 1.00)	0.53 (0.23 to 0.84)
LightGBM	0.04 (0.00 to 0.10)	0.30 (0.00 to 0.63)	0.13 (0.01 to 0.35)
Isolation forest	0.06 (0.01 to 0.12)	0.40 (0.10 to 0.73)	0.30 (0.02 to 0.60)
ECOD	0.07 (0.02 to 0.14)	0.50 (0.17 to 0.83)	0.13 (0.03 to 0.38)

Measured on the 10 errors between 2026-07-13 and 2026-10-14, trained on the 25 errors before them. Each figure carries its 95% interval: a precision read off ten events cannot separate two models, and here none of the intervals do.

What this cannot claim

35 errors is a small number, and saying so is part of the work.

- A time split leaves about a dozen errors to be tested on. Any precision measured on a dozen events carries a wide interval, so **every figure is published with its interval**, and two models whose intervals overlap are reported as not separated.
- 24 features against 35 errors is far past the usual rule of ten events per feature. The feature set is cut, and the cut is reported.
- The invoices are simulated and the simulation is known. More of them would narrow the intervals **around the simulator**, not around a real supplier.

So the project makes two separate claims, and keeps them apart: that the system operates, shown by it running and logging; and that the model finds errors, bounded by data that is not real.

How to run it

```
python -m pip install -r requirements.txt
python -m pytest -q
python run.py prepare        load the run, build and cut the features
python run.py train          split by time, fit every model, report
python run.py score          write the rows the dashboard reads
python run.py watch          walk the run week by week, and keep going
python run.py serve          keep the gauges up
```

Every command appends one JSON line to `data/runs.log`, so days of running stay readable.

The files

```
02-invoice-anomalies/
â”œâ”€â”€ run.py                  - the commands
â”œâ”€â”€ src/
â”‚   â”œâ”€â”€ data/load.py        - the run, as typed tables
â”‚   â”œâ”€â”€ features/build.py   - the features, none from the future
â”‚   â”œâ”€â”€ evaluation/
â”‚   â”‚   â”œâ”€â”€ metrics.py      - confusion matrix, precision, recall, F1
â”‚   â”‚   â”œâ”€â”€ intervals.py    - the interval around every figure
â”‚   â”‚   â”œâ”€â”€ reading.py      - perfect data against what was read
â”‚   â”‚   â”œâ”€â”€ splits.py       - by time, labels that arrive late
â”‚   â”‚   â””â””â”€â”€ watch.py        - weekly precision, drift
â”‚   â”œâ”€â”€ models/
â”‚   â”‚   â”œâ”€â”€ exporter.py     - the figures the dashboard reads
â”‚   â”‚   â”œâ”€â”€ feedback.py     - the verdict as a label
â”‚   â”‚   â”œâ”€â”€ lifecycle.py    - retrain, challenge, promote
â”‚   â”‚   â”œâ”€â”€ rules.py        - the clerk's checks, the bar to beat
â”‚   â”‚   â”œâ”€â”€ scoring.py      - scores and decisions
â”‚   â”‚   â””â””â”€â”€ train.py        - the models, recorded in MLflow
â”‚   â””â””â”€â”€ runs/loop.py        - the loop, week by week
â””â””â”€â”€ tests/                  - the same folders
```